



Protocol Audit Report

Version 1.0

Joachim

October 8, 2026

Protocol Audit Report

Joachim

October 8 2026

Prepared by: Joachim Lead Security Researcher: - Joachim

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [#-1] Storing the password on-chain makes it visible to anyone, and no longer private
 - * [#-2] `PasswordStore::setPassword` has no access controls, meaning a non-owner could change the password
 - Informational
 - * [I-1] The `PasswordStore::getPassword` natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect

Protocol Summary

PasswordStore is a protocol dedicated to storage and retrieval of a user's passwords. The protocol is designed to be used by a single user, and is not designed to be used by multiple. Only the owner should be able to set and access this password.

Disclaimer

Joachim made all effort to find many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by Joachim is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond with the following commit hash:

```
1 7d55682ddc4301a7b13ae9413095feffd9924566
```

Scope

- Commit Hash: 2e8f81e263b3a9d18fab4fb5c46805ffc10a9990

- In Scope:

```
1 ./src/  
2 #-- PasswordStore.sol
```

Roles

- Owner: The user who can set the password and read the password.
- outsiders: No one else should be able to set or read the password.

Executive Summary

- We spent 1 week with Patrick Collins, doing a manual review and carrying out some tests.*

Issues found

Severity	Number of issues found
High	2
Medium	0
Low	0
Info	1
Total	3

Findings

High

[#-1] Storing the password on-chain makes it visible to anyone, and no longer private

Description: All data stored on-chain is visible to anyone and can be read directly from the blockchain. The `PasswordStore::s_password` variable is intended to be private and accessible only through the `PasswordStore::getPassword` function, which is intended to be called only by the owner of the contract.

We show one method of reading on-chain data below.

Impact: Anyone can read the private password, severely breaking the functionality of the protocol.

Proof of Concept:

The steps below show how anyone can read the password directly from the blockchain.

1. Create a locally running chain:

```
1 make anvil
```

- ## 2. Deploy the contract to the chain:

```
1 make deploy
```

3. Run the storage tool. We use 1 because that is the storage slot of `s_password` in the contract.

```
1 cast storage <ADDRESS_HERE> 1 --rpc-url http://127.0.0.1:8545
```

You will get an output that looks like this:

[illegible]

You can then parse that hex to a string with:

[illegible]

And get an output of:

```
1 myPassword
```

Recommended Mitigation: Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain and then store the encrypted password on-chain. This would require the user to remember another password off-chain to decrypt it. However, you would also likely want to remove the view function, as you would not want the user to accidentally send a transaction with the password that decrypts your password.

[-2] PasswordStore::setPassword has no access controls, meaning a non-owner could change the password

Description: The `PasswordStore : setPassword` function is set to be `external`; however, the natspec of the function and the overall purpose of the smart contract state that `This function allows only the owner to set a new password.`

```
1 function setPassword(string memory newPassword) external {
2     // @audit - There are no access controls
3     s_password = newPassword;
4     emit SetNetPassword();
5 }
```

Impact: Anyone can set or change the password of the contract, severely breaking the contract's intended functionality.

Proof of Concept: Add the following to the `PasswordStore.t.sol` test file.

Code

```
1 function test_anyone_can_set_password(address randomAddress) public {
2     vm.assume(randomAddress != owner);
3     vm.prank(randomAddress);
4     string memory expectedPassword = "myNewPassword";
5     passwordStore.setPassword(expectedPassword);
6
7     vm.prank(owner);
8     string memory actualPassword = passwordStore.getPassword();
9     assertEq(actualPassword, expectedPassword);
10 }
```

Recommended Mitigation: Add an access control conditional to the `setPassword` function.

```
1 if (msg.sender != i_owner) {
2     revert PasswordStore__NotOwner();
3 }
```

Informational

[I-1] The PasswordStore::getPassword natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect

Description: The natspec documents a `newPassword` parameter, but `getPassword` takes no parameters.

```
1 /*
2  * @notice This allows only the owner to retrieve the password.
3  * @audit there is no newPassword parameter
4  * @param newPassword The new password to set.
5  */
6 function getPassword() external view returns (string memory) {
7     if (msg.sender != i_owner) {
8         revert PasswordStore__NotOwner();
9     }
```

```
10     return s_password;  
11 }
```

Impact: The natspec is incorrect.

Recommended Mitigation: Remove the incorrect natspec line.

```
1 - * @param newPassword The new password to set.
```